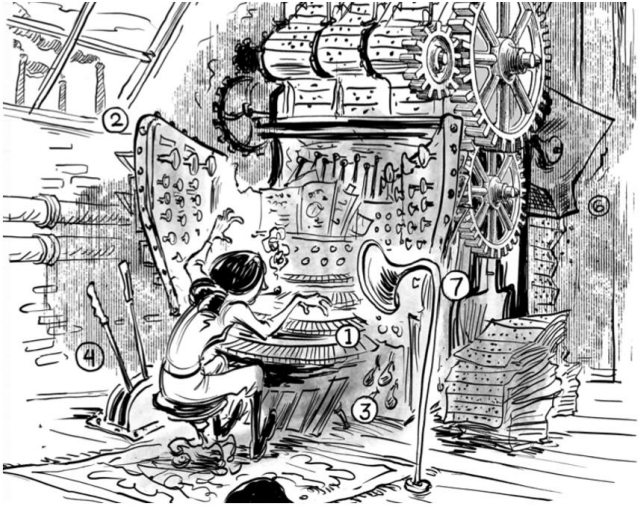
University of Exeter

COMM2020 Team Project

Overview + Team working/Roles + Requirement Elicitation, Analysis & Specification

Prof. Solomon Oyelere
Department of Computer Science
s.oyelere@exeter.ac.uk

Vision



University of Exeter

COM2020

Team Project

Teams apply technical skills in software development and data analysis while managing project planning, teamwork, and communication.

COM2020 Team Project

Lecture: Introduction to the project + Team working/Roles + Requirement Elicitation, Analysis & Specification

What you're building

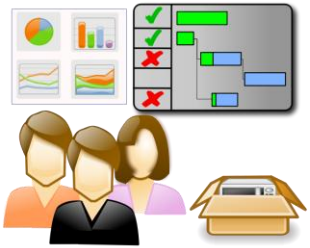
How you'll be assessed

How teams succeed

Introduction

In this module you will:

- Solve a substantial practical problem from a computational and data-based perspective
- Apply software development + data analysis skills
- Balance innovation with feasibility
- Build a prototype and demonstrate success using clear measures
- Practice planning, communication, and teamwork like in industry



Think "mini product team"

Problem → plan → build → evaluate → demo

What does that look like?

University of Exeter

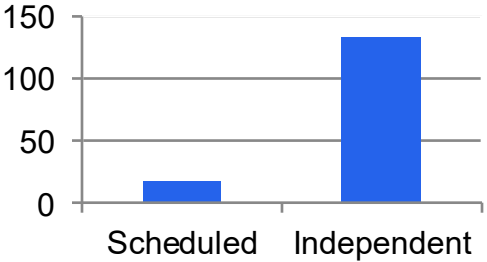
- Be capable of problem solving with teams
  - Be able to **decompose a problem for teams**
  - Be able to recognise and adopt **roles**
- Apply an **integrated or systems** approach to the solution of complex problems.
- Apply knowledge of domain context, project and change management, and relevant legal matters including intellectual property rights.
- Select and apply appropriate **materials, technologies, and processes**, and recognise their limitations.
- Enjoy the **experience**

How the module runs

- 11-week team project
- Weekly clinic with TAs & Convenor
- Early lectures cover teamwork, project management, etc.
- Work iteratively: build, test, evaluate, improve
- 2 demos/presentation of your prototype

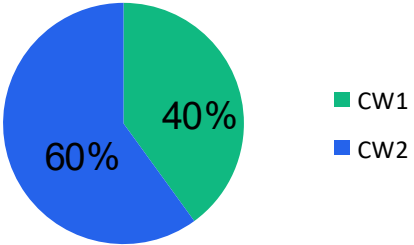
Workload & assessment

Study time split (hours)



Most of the work happens outside timetabled sessions.

Summative assessment weighting



Treat CW1 as your foundation: scope, plan, early prototype.

Team Preferences

Groups of 7-8 students – Complete your group preferences via <u>this form</u> ***by Tomorrow,*** Tuesday, 13th January

The team projects will be explained in this weeks' engagement and support clinic.

Team working/roles

Team working/roles

University of Exeter

Typical roles

Project lead
Plans, facilitates, keeps risks visible

Technical lead
Architecture, integration decisions, code quality

Data/ML lead
Data pipeline, modelling choices, evaluation design

UI/UX designers
Creating user experience, Stakeholders, user stories, acceptance criteria

Software developer
Build & maintain software

DevOps engineer
Bridges Dev & Ops, focusing on automation, deployment, & infrastructure

QA & testing
Test strategy, bug triage, definition of done

Documentation & comms
Demo narrative, report, decisions log

Aim for shared ownership: rotate roles where it helps learning and resilience.

Team working/roles

University of Exeter

Good practice

- Every task has an owner
- Roles ≠ job titles — they are responsibilities
- Rotate where safe to grow skills
- Keep decisions visible (short logs)
- Be explicit about inclusion & communication norms

Make responsibilities explicit

Legend: R = Responsible A = Accountable C = Consulted I = Informed

Task / Deliverable	Proj lead	Tech	Data	QA	Docs
Define success measures	A	C	R	C	I
Requirements & scope	A	C	C	I	R
Prototype build	C	A/R	R	C	I
Evaluation report	C	C	A/R	R	I
Demo + final report	A	C	C	I	R

Tip: keep it simple. One “A” per row is often enough, and “R” can be shared.

Working agreements

University of Exeter

Key components of a successful team project

Shared goals & time plan
Weekly plan + 2hrs clinic

Communication
One main channel, clear response expectations

Definition of done
Code + tests + docs + review

Decision-making
Timebox, then decide and record rationale

Inclusivity
No interrupting; rotate facilitation; invite quieter voices

Risk & scope
Keep a “nice-to-have” list so core stays safe

Teams change over time

University of Exeter

Tuckman's stages of group development

Forming
Polite, unsure about roles

Storming
Disagreements & stress

Norming
Shared norms & flow

Performing
High trust + delivery

Adjourning
Wrap-up & reflect

When conflict happens

- Name the problem, not the person
- Use evidence: tests, data, acceptance criteria
- Timebox debates; decide and move forward
- Escalate early to your supervisor if stuck

Low-effort routines that help

- Weekly retro: keep / stop / start
- Short decision log (1–2 lines each)
- Visible task board with owners
- Pair on tricky tasks to reduce knowledge silos

The Overriding Objective

The overriding objective of this module is to have fun! Remember this!

Team project – Requirement elicitation, analysis & specification

What is Team Project?

*a **team project** is a collaborative endeavor where a group of individuals works together toward a common goal or a specific set of deliverables within a defined timeframe.*

Engineering Principles

The outcome of your product is …

Testability
Maintainability
Integrity
External integration is well defined
Ethics
Management

Project development phases

1. the initiation, feasibility and concept
2. the planning, requirements capture and design phase
3. the execution/implementation phase
4. the monitoring and control
5. the closure and handover
6. the post-project optimization phase

The Initiation Phase

- Clarify the **problem** and who the system is for (user, client, stakeholder).
- Define the **data context**: what data exists, where it comes from, access constraints, ethics/privacy.
- Decide what “success” means: accuracy targets, runtime, usability, reliability, or business impact.
- Outputs: problem statement, initial scope, assumptions, stakeholders, feasibility notes, early risks (e.g., data availability).

How do we clarify the problem?

The Planning Phase

- Translate the goal into **requirements** (functional + non-functional).
- Plan the **architecture** (components, APIs, storage, model/service boundaries).
- Create a **work breakdown**: backlog/user stories, sprint plan, milestones, dependency map.
- Plan **data work**: collection, cleaning, feature engineering, labelling, split strategy, evaluation metrics.
- Outputs: requirements spec, architecture diagram, backlog, timeline, risk register, evaluation plan.

The Execution Phase

- Build in increments: set up repo, Continuous Integration environments, then implement features iteratively.
- Develop the **data pipeline** (ingestion → cleaning → transformation → storage).
- Implement algorithms/models and the application layer (frontend, backend, services).
- Outputs: working code increments, integrated components, data pipeline, baseline model/analytics, documentation updates.

The Monitoring & Control Phase

- Track progress and quality continuously:
  - **Engineering:** unit/integration tests, CI status, code reviews, performance profiling.
  - **Data:** data quality checks, schema drift, missingness, leakage checks.
  - **Model/analytics (if relevant):** validation metrics, bias checks, overfitting monitoring.
- Manage changes: scope creep, new requirements, unexpected data issues.
- Outputs: sprint reviews, updated backlog, test reports, metric dashboards, resolved issues, controlled change decisions.

The Closure Phase

- Stabilise and package the final deliverable:
  - freeze features, fix critical bugs, final integration
  - prepare deployment or runnable release (Docker/image, installer, or reproducible runbook)
  - final documentation: how to run, configure, and test; known limitations
- Outputs: final release/tag, deployment package or reproducible demo, final report, user/admin guide, handover.

The Evaluation Phase

- Measure outcomes against the original success criteria:
  - does it meet performance, correctness, usability, robustness, and maintainability needs?
  - compare against baselines (simple heuristic, earlier version, existing tool)
- Reflect on process and teamwork:
  - what technical decisions paid off, what caused rework, what you'd prioritise next iteration
- Outputs: evaluation results, lessons learned/retrospective, future work plan, ethics and limitations statement.

Short project planning exercise

Take 3 mins break

Week 1 checklist

If you only do one thing: pick a tiny demo and build towards it.

- Confirm team members and communication channels
- Choose initial roles (and agree what “role rotation” means)
- Create a shared task board and a decision log
- Write a one-paragraph problem statement and a success measure
- Identify risks/constraints (time, data, skills) and reduce scope
- Plan your first iteration (1–2 weeks) with a demo goal
- Come to weekly supervisor meeting (during clinics) and come with questions

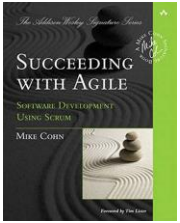
References

H. Van Vliet, Software Engineering: Principles and Practice, John Wiley, 2008, ISBN 978-0470031469.

I. Sommerville, Software Engineering, Pearson, 2010, ISBN 978- 0137053469.

R. S. Pressman, Software Engineering: A Practitioner’s Approach, McGraw-Hill, 2009, ISBN 978-0071267823.

References



M. Cohn, Succeeding With Agile: Software Development Using Scrum, Pearson, 2010, ISBN 978-0321579362.